

Journal de bord — ReadNet Flutter

Ce que j'ai construit, comment ça fonctionne, et ce que j'ai appris en le faisant.

Le contexte

ReadNet est une application mobile utilisée au Sénégal pour évaluer les compétences en lecture des enfants, selon la méthode ASER (Annual Status of Education Report). Il existait déjà une version Android en Java. Mon travail : la reconstruire en Flutter en apprenant au fur et à mesure.

Je n'avais pas une connaissance avancée de Flutter au départ. Ce document retrace ce que j'ai construit, les choix techniques que j'ai faits, et surtout les erreurs qui m'ont appris des choses.

Ce qu'on a installé et pourquoi

Flutter + Dart. Le choix de base. Une seule codebase pour Android (et iOS si besoin plus tard). Dart est un langage typé, proche de Java ou TypeScript, donc pas trop dépayçant.

Riverpod pour la gestion d'état. Il y a aussi Provider, Bloc, GetX... J'ai choisi Riverpod parce qu'il force à être explicite sur ce qui est partagé dans l'app. Un **Provider** Riverpod c'est un objet global accessible depuis n'importe quel widget sans passer par le constructeur à chaque fois.

Drift pour la base de données locale. Drift est un ORM au-dessus de SQLite. L'idée : tu décris tes tables en Dart, tu lances une commande, et il génère tout le code de requête automatiquement. C'est beaucoup plus propre qu'écrire des chaînes SQL à la main.

go_router pour la navigation. Flutter a une navigation native mais elle devient vite compliquée dès qu'on veut des routes nommées ou passer des données entre écrans. go_router ressemble à ce qu'on ferait avec React Router.

Dio pour les appels HTTP. L'équivalent de Retrofit en Java ou Axios en JavaScript. Gère les timeouts, les headers, l'upload multipart.

record pour enregistrer l'audio. L'enfant lit à voix haute, l'évaluateur enregistre. Simple.

archive pour créer des fichiers ZIP. On emballe les enregistrements audio + un fichier JSON dans un ZIP avant d'envoyer au serveur.

La structure du projet

J'ai organisé le projet par fonctionnalité (features), pas par type de fichier. C'est ce qu'on appelle "feature-first" :

```
lib/  
├─ core/           → thème, routes, constantes API  
├─ data/
```

```

├── database/    → tables Drift + AppDatabase + provider
├── models/      → modèles pour les réponses API
├── services/    → logique réseau (pull données, sync upload)
├── features/
│   ├── auth/    → écran de connexion
│   ├── home/    → écran principal (3 onglets)
│   ├── assessment/ → flux d'évaluation
│   └── pull_crl/ → premier lancement, téléchargement des données
├── shared/
│   └── widgets/  → composants réutilisables

```

Pourquoi cette structure ? Parce que quand je cherche "tout ce qui touche à l'évaluation", je vais dans `features/assessment/`. Je n'ai pas à passer entre `screens/`, `controllers/`, `models/` dans des dossiers séparés.

La base de données avec Drift

Comment Drift fonctionne

Tu crées une classe par table qui étend `Table` :

```

class AserToolTable extends Table {
  TextColumn get id => text();
  TextColumn get data => text();
  TextColumn get type => text();
  TextColumn get language => text();
  IntColumn get sampleNumber => integer().withDefault(const Constant(1))();
  BoolColumn get isAttempted => boolean().withDefault(const Constant(false))();

  @override
  Set<Column> get primaryKey => {id};
}

```

Ensuite tu lances `flutter pub run build_runner build` et Drift génère un fichier `.g.dart` avec toute la logique de requête. Tu n'écris jamais de SQL.

Pour interroger la base :

```

Future<List<AserToolTableData>> getAserToolItems(
  String type, String language, int limit) {
  return (select(aserToolTable)
    ..where((t) =>
      t.type.equals(type) &
      t.language.equals(language) &
      t.isAttempted.equals(false))
    ..orderBy([(t) => OrderingTerm.asc(t.sampleNumber)])
    ..limit(limit))

```

```
        .get();  
    }
```

Ça ressemble à du SQL mais c'est du Dart pur. Le compilateur vérifie les types.

Les migrations

Quand tu ajoutes une table après avoir déjà installé l'app, SQLite ne la crée pas automatiquement. Il faut gérer les migrations :

```
@override  
int get schemaVersion => 8;  
  
@override  
MigrationStrategy get migration => MigrationStrategy(  
    onCreate: (m) => m.createAll(), // première installation  
    onUpgrade: (m, from, to) async {  
        if (from < 2) await m.createTable(municipalityTable);  
        if (from < 3) await m.createTable(instituteTable);  
        // ...  
    },  
);
```

À chaque fois que tu ajoutes une table, tu incrémente `schemaVersion` et tu ajoutes une condition. Si quelqu'un a l'app depuis la version 3 et passe à la version 8, toutes les conditions entre 3 et 8 s'exécutent dans l'ordre.

Le provider Riverpod pour la base

Pour que n'importe quel écran puisse accéder à la base, on crée un provider :

```
final databaseProvider = Provider<AppDatabase>((ref) {  
    final db = AppDatabase();  
    ref.onDispose(db.close);  
    return db;  
});
```

`ref.onDispose` ferme proprement la connexion quand le provider est détruit. Dans les widgets, on y accède avec `ref.read(databaseProvider)`.

La navigation avec go_router

go_router fonctionne avec des routes nommées comme des URLs :

```
final appRouter = GoRouter(  
  initialLocation: '/login',  
  routes: [  
    GoRoute(path: '/login', builder: (_, __) => const LoginScreen()),  
    GoRoute(path: '/home', builder: (_, __) => const HomeScreen()),  
    GoRoute(  
      path: '/assessment/child',  
      builder: (context, state) => SelectChildScreen(  
        schoolId: state.extra as String,  
      ),  
    ),  
  ],  
);
```

Pour naviguer en passant des données :

```
context.push('/assessment/child', extra: schoolId);
```

`state.extra` récupère ce qu'on a passé. Le `as String` est un cast — si tu passes autre chose qu'un String, tu auras une erreur à l'exécution. C'est utile pour les données simples comme des IDs.

La différence entre `context.go()` et `context.push()` : `go()` remplace toute la pile de navigation (comme rediriger), `push()` empile par-dessus (comme ouvrir une nouvelle page).

L'authentification

L'écran de connexion vérifie les identifiants contre la table `Cr1Table` en local :

```
Future<void> _onLogin() async {  
  final db = ref.read(databaseProvider);  
  final cr1 = await db.checkCredentials(username, password);  
  
  if (cr1 != null) {  
    final prefs = await SharedPreferences.getInstance();  
    await prefs.setString('cr1Id', cr1.cr1Id);  
    await prefs.setString('cr1Name', '${cr1.firstName} ${cr1.lastName ?? ''}');  
    context.go('/home');  
  } else {  
    _showError('Identifiants incorrects.');
```

SharedPreferences c'est un stockage clé-valeur persistant. On y sauvegarde l'ID de l'utilisateur connecté pour s'en souvenir sans rouvrir la base à chaque fois.

L'animation de secousse

Quand les identifiants sont faux, la carte se secoue. C'est fait avec un `AnimationController` et `math.sin()` :

```
_shakeController = AnimationController(  
  vsync: this,  
  duration: const Duration(milliseconds: 500),  
);  
  
// Dans le build :  
final offset = math.sin(_shakeAnimation.value * math.pi * 4) * 8;  
return Transform.translate(offset: Offset(offset, 0), child: child);
```

`math.sin()` génère une oscillation entre -1 et 1. On multiplie par 8 pour avoir des pixels. Le `* 4` accélère les cycles pour que ça fasse 4 allers-retours en 500ms.

Le téléchargement initial des données

Avant de pouvoir travailler, l'évaluateur doit télécharger les données depuis le serveur : son compte, les communes, les établissements, les écoles, les enfants, et les outils de test.

`PullCrlService` fait tous ces appels API avec Dio :

```
final response = await _dio.get(ApiConstants.getAserSamples);  
final List data = response.data['data'];
```

`response.data` contient directement l'objet Dart parsé depuis le JSON. Dio s'occupe de la désérialisation.

Chaque item est ensuite inséré en base avec `insertOnConflictUpdate` — si l'item existe déjà, il est mis à jour, sinon créé. C'est l'équivalent d'un `UPSERT` en SQL.

Un bug qui m'a pris du temps

L'API retourne le type `"Non-word"` avec un `w` minuscule. J'avais codé `"Non-Word"` avec un `W` majuscule dans mon app. Résultat : zéro items chargés pour ce niveau, sans aucune erreur. Ce genre de bug est invisible parce qu'il ne plante pas, il retourne juste une liste vide.

La leçon : toujours vérifier les données brutes de l'API avec un outil comme curl ou Postman avant d'écrire le code qui les consomme.

Le flux d'évaluation

Le flux de navigation pour commencer une évaluation :

Commune → Établissement → École → Enfant → Test

Chaque écran reçoit l'ID du niveau supérieur via `state.extra` et charge ses données depuis la base locale.

Sélection de l'enfant et création de session

Sur l'écran de sélection de l'enfant, on choisit la langue du test (Wolof, Peulh, Serere), et on crée une session :

```
final examId =
'${crlId}_${student.studentId}_${DateTime.now().millisecondsSinceEpoch}';

await db.insertSession(StudentSessionTableCompanion(
  examId: Value(examId),
  studId: Value(student.studentId),
  crlId: Value(crlId),
  childFullName: Value(student.studentName),
  language: Value(selectedLanguage),
  startTime: Value(DateTime.now().toIso8601String()),
));

await prefs.setString('currentLevel', 'Letter');
context.push('/assessment/test', extra: examId);
```

L'`examId` est une clé unique composée de l'ID du CRL + ID de l'enfant + timestamp. On le sauvegarde en base et on le passe à l'écran de test.

L'écran de test

C'est l'écran le plus complexe. Il gère 4 niveaux dans l'ordre :

```
const _levelOrder = ['Letter', 'Word', 'Non-word', 'Passage'];
const _levelLimits = {'Letter': 9, 'Word': 11, 'Non-word': 11, 'Passage': 1};
```

Pour chaque niveau, on charge les items depuis la base, on les affiche un par un, l'évaluateur enregistre la lecture de l'enfant, puis note la réponse.

Le système de notation

Trois valeurs possibles :

- 2 → correct
- 1 → faux
- 9 → pas de réponse

En base, on stocke `-1` comme valeur initiale (non tenté). Quand on récupère les réponses pour calculer le score, on filtre sur `isAttempted = true`.

L'enregistrement audio

```
final recorder = Record();

Future<void> _startRecording(String path) async {
  await recorder.start(path: path, encoder: AudioEncoder.aacLc);
}

Future<void> _stopRecording() async {
  await recorder.stop();
}
```

Le chemin du fichier audio : `{documentsDir}/{examId}/{examId}_{itemId}.m4a`

On sauvegarde ce chemin dans la base avec la réponse. Plus tard, quand on crée le ZIP pour l'upload, on retrouve le fichier par son chemin.

Un plugin qui ne fonctionnait pas

Après avoir ajouté `record` dans `pubspec.yaml`, j'obtenais `MissingPluginException` au lancement. Le plugin natif Android n'était pas enregistré.

Solution : désinstaller complètement l'app du téléphone et réinstaller. Flutter n'effectue pas toujours une réinstallation complète lors du hot reload ou même d'un simple `flutter run` si l'app était déjà installée.

```
adb uninstall com.baamtu.pratham_clone
flutter run
```

La progression entre niveaux

Quand tous les items d'un niveau sont traités, on passe au suivant. La méthode `_nextLevel()` recharge les items du prochain niveau depuis la base et réinitialise l'index courant. Si on est au dernier niveau, on clôture la session.

La synchronisation avec le serveur

Construction du ZIP

Pour chaque session à envoyer, on crée un ZIP qui contient :

- Un fichier JSON avec les métadonnées de la session et toutes les réponses
- Les fichiers audio M4A correspondant à chaque enregistrement

```
final encoder = ZipEncoder();
final archive = Archive();

final jsonBytes = utf8.encode(jsonEncode(sessionJson));
archive.addFile(
  ArchiveFile('$examId/${examId}INFO.json', jsonBytes.length, jsonBytes),
);

for (final answer in answers) {
  final audioFile = File(answer.recordingName!);
  if (!await audioFile.exists()) continue;
  final audioBytes = await audioFile.readAsBytes();
  final filename = audioFile.path.split('/').last;
  archive.addFile(
    ArchiveFile('$examId/$filename', audioBytes.length, audioBytes),
  );
}

return encoder.encode(archive)!;
```

L'upload multipart

Le ZIP est envoyé comme fichier dans un formulaire multipart :

```
final formData = FormData.fromMap({
  'zipfile': await MultipartFile.fromFile(
    zipFile.path,
    filename: '${session.examId}.zip',
  ),
});

final response = await _dio.post(
  ApiConstants.pushZipFiles,
  data: formData,
);
```

Si le serveur répond avec `status: 'success'`, on marque la session comme envoyée en base pour ne pas la renvoyer.

Le design de l'interface

Le thème global

Tout le thème est défini dans `lib/core/theme.dart`. On y déclare les couleurs :

```
class AppColors {
  static const primary = Color(0xFF0362A0);
```



```
static const error = Color(0xFFD0021B);
static const textDark = Color(0xFF292929);
// ...
}
```

Et le `ThemeData` de `MaterialApp` avec les styles par défaut des boutons, champs de texte, cartes. L'avantage : on change la couleur primaire à un seul endroit et ça se propage partout.

Le fond de l'écran de connexion

Un `Stack` avec plusieurs couches :

1. Un `Container` avec `LinearGradient` (bleu foncé vers bleu clair)
2. Des `Container` en cercles positionnés avec `Positioned` pour la décoration
3. Un `CustomPainter` qui dessine une grille de petits points

`CustomPainter` c'est la façon de dessiner librement sur le canvas en Flutter. Il suffit d'implémenter la méthode `paint(Canvas canvas, Size size)`.

Les champs de saisie animés

Chaque champ de texte est enveloppé dans un `AnimatedContainer` qui change de bordure et de couleur de fond selon le focus :

```
Focus(
  onFocusChange: (focused) => setState(() => _isFocused = focused),
  child: AnimatedContainer(
    duration: const Duration(milliseconds: 200),
    decoration: BoxDecoration(
      border: Border.all(
        color: _isFocused ? AppColors.primary : AppColors.border,
        width: _isFocused ? 2 : 1.5,
      ),
    ),
    child: TextField(...),
  ),
)
```

`AnimatedContainer` interpole automatiquement entre les valeurs de décoration quand elles changent. Pas besoin d'`AnimationController`.

Le texte en dégradé

Pour le titre "ReadNet" avec un dégradé de couleur :

```
ShaderMask(
  shaderCallback: (bounds) => const LinearGradient(
    colors: [Color(0xFF0362A0), Color(0xFF0484D8)],
  ).createShader(bounds),
```

```
child: Text(  
  'ReadNet',  
  style: TextStyle(color: Colors.white), // le blanc est remplacé par le shader  
)  
)
```

ShaderMask applique un shader graphique sur son enfant. La couleur `Colors.white` du texte sert de "masque" — le shader remplace le blanc par le dégradé.

Les galères qu'on a rencontrées

Version de `record` incompatible. La version 5.x du package `record` avait cassé son API sur Linux. On a rétrogradé à la version 4.4.4 et changé `AudioRecorder()` en `Record()`.

Un seul item affiché par niveau. La requête Drift avait un filtre `sampleNumber.equals(1)` hérité d'une ancienne logique. Elle retournait toujours le premier item au lieu de tous les items. Suppression du filtre + ajout d'un `limit` en paramètre.

Overflow sur l'écran de connexion. Le texte "Première utilisation ? S'enregistrer" dépassait la largeur disponible. Solution : `width: double.infinity` sur le container + `Flexible` + `TextOverflow.ellipsis` sur le texte.

`path_provider` déclaré deux fois dans `pubspec.yaml`. Le package `record` l'inclut déjà comme dépendance. Déclarer les deux créait un conflit de version. Suppression de la ligne dupliquée.

Ce que je retiens

Flutter n'est pas compliqué mais il a sa propre logique. Les points qui m'ont le plus pris de temps à comprendre :

Les widgets sont immuables. Un `StatelessWidget` ne peut pas changer après sa création. Pour un état qui change, on utilise `StatefulWidget` avec `setState()`. Pour un état partagé entre plusieurs écrans, on utilise Riverpod.

`build_runner` est obligatoire pour Drift. Chaque modification de table nécessite de relancer `flutter pub run build_runner build`. Si tu oublies, le code compilé ne correspond plus aux tables définies.

Le hot reload ne réinstalle pas les plugins natifs. Quand tu ajoutes un package qui a du code natif Android (comme `record`), il faut désinstaller et réinstaller l'app complètement.

Les types de l'API sont des contrats. Si l'API retourne `"Non-word"` et que tu codes `"Non-Word"`, zéro résultat sans erreur. Toujours vérifier les données brutes.

Projet : ReadNet Flutter — Sénégal 2024